Maximum likelihood estimation

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

In this notebook, we study **maximum likelihood estimation** of the *Student t* distribution. In the case of this distribution, the maximum likelihood estimate cannot be solved analytically, but instead numerical optimisation must be used.

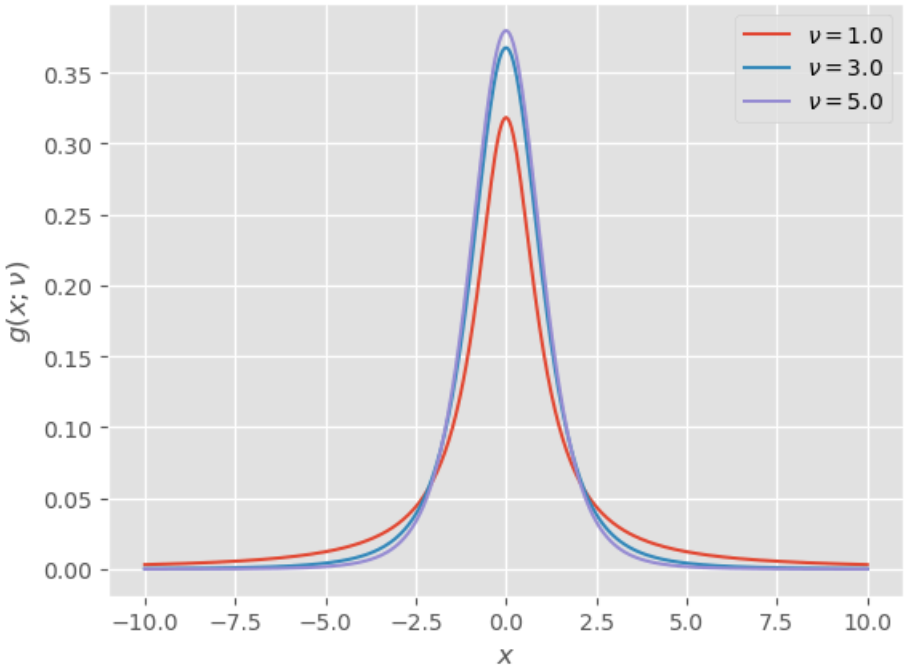
In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import minimize
from scipy.stats import t

plt.style.use("ggplot")
```

The pdf of the Student t distribution with $\nu > 0$ degrees of freedom is

$$g(x;\nu) = C_\nu \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \quad x \in \mathbb{R},$$

where $C_\nu > 0$ is a normalising constant (but otherwise inconsequential). The module `t` from `scipy.stats` provides this pdf:

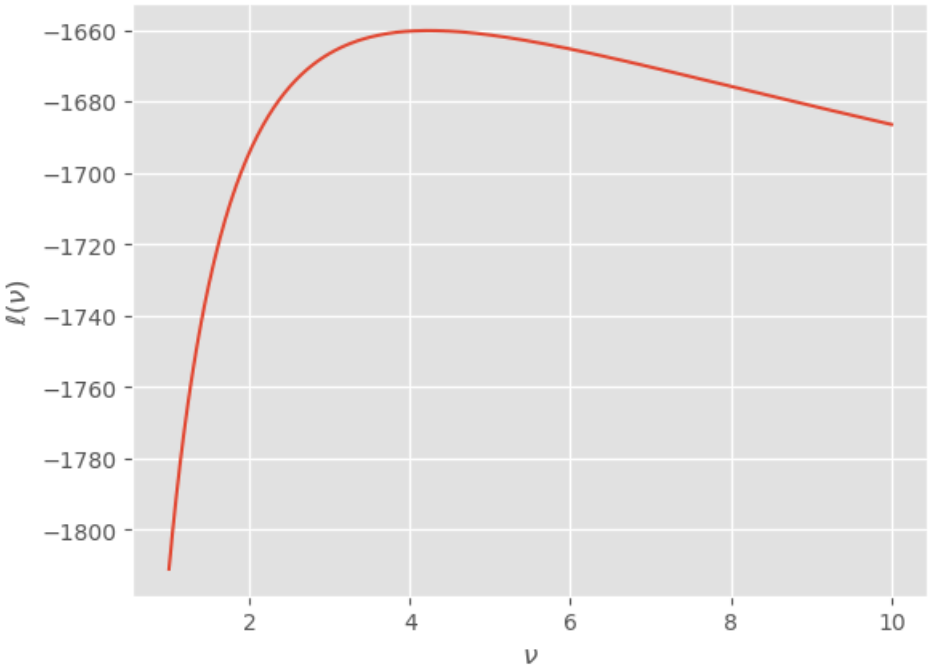
In [2]:
```
x = np.linspace(-10.0, 10.0, 401)
plt.plot(x, t.pdf(x, 1.0), label=r"$\nu = 1.0$")
plt.plot(x, t.pdf(x, 3.0), label=r"$\nu = 3.0$")
plt.plot(x, t.pdf(x, 5.0), label=r"$\nu = 5.0$")
plt.xlabel(r"$x$")
plt.ylabel(r"$g(x;\nu)$")
plt.legend()
plt.show()
```



Let us simulate 1000 observations from the distribution with $\nu = 4$ and define the log-likelihood function:

In [3]:
```
def log_likelihood(dfs, data):
    dfs = np.atleast_1d(dfs)
    data = np.asarray(data)
    return np.array([t.logpdf(data, df) for df in dfs]).sum(axis=1)

data = t.rvs(4.0, size=1000, random_state=123)
```

```
dfs = np.linspace(1.0, 10.0, 401)
plt.plot(dfs, log_likelihood(dfs, data))
plt.xlabel(r"$\nu$")
plt.ylabel(r"$\ell(\nu)$")
plt.show()
```



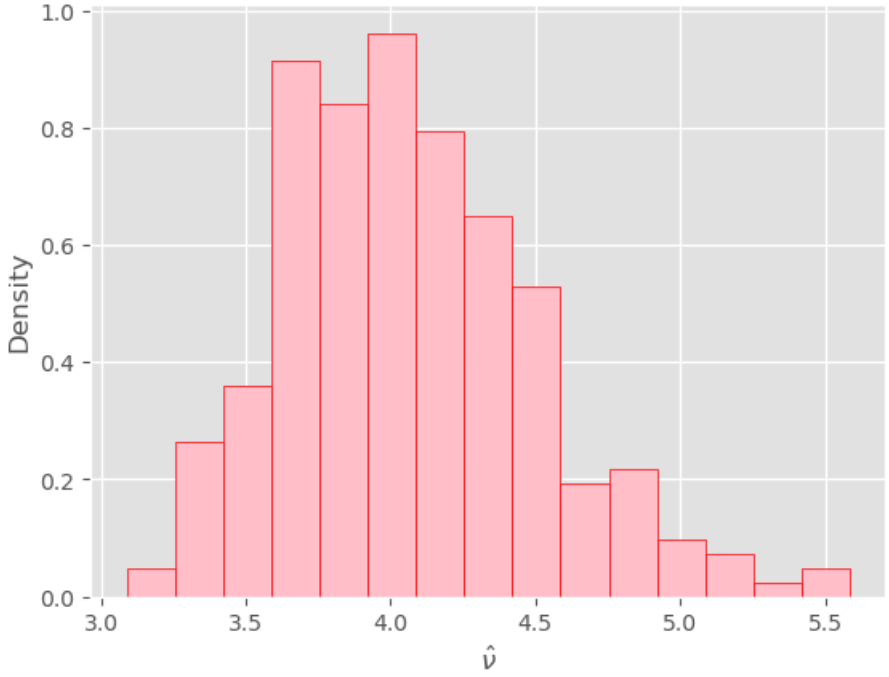
While the maximiser could be found visually, let us try to maximise the log-likelihood numerically:

In [4]:
```
# define negative log-likelihood since the optimisation function performs minimisation
neg_log_likelihood = lambda df: -log_likelihood(df[0], data)
# draw initial value from Exp(1) distribution
np.random.seed(123)
df_init = np.random.exponential(1.0)

result = minimize(
    fun=neg_log_likelihood, x0=df_init, method="L-BFGS-B", bounds=[(1e-3, None)]
)

print(result.x[0])
```

```
4.23787818776195
```

Let us then repeat this 250 times to see how well the estimates concentrate around $\nu = 4$:

In [5]:
```
n_reps = 250
df_hat = []

for i in range(n_reps):
    data = t.rvs(4.0, size=1000, random_state=i)
    neg_log_likelihood = lambda df: -log_likelihood(df[0], data)

    result = minimize(
        fun=neg_log_likelihood, x0=df_init, method="L-BFGS-B", bounds=[(1e-3, None)]
    )

    df_hat.append(result.x[0])

plt.hist(df_hat, bins=15, density=True, color="pink", edgecolor="red")
plt.xlabel(r"$\hat{\nu}$")
plt.ylabel("Density")
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
Density
3.0
3.5
4.0
4.5
5.0
5.5
$\hat{\nu}$